

The Mental Model.

A living, daily-updated map of every concept learned — built to be skimmed in five minutes and remembered for good.



Java + Spring Boot



Google Cloud Bigtable



GKE Deployment

DAY 4 OF 17

DEADLINE: SEP 25, 2026



Ankit · 9 yrs Data Engineering

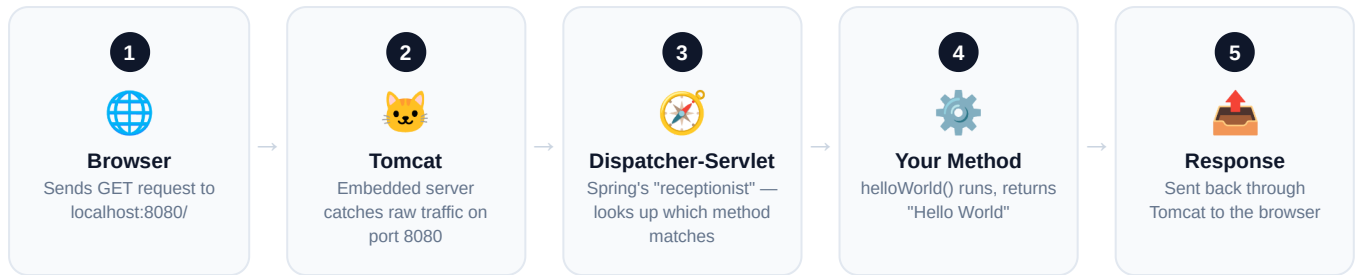
Spark · Python · SQL · BigQuery · Composer → now learning Java from zero

OUT-OF-DOMAIN SKILLS TEST

Pivoted Sept 8, 2026

How one HTTP request travels through Spring Boot

The picture to keep in your head every time you see `@RestController` or `@GetMapping`.



Spring builds its lookup table (step 3) once, at startup — by scanning every class tagged `@RestController` and every method tagged `@GetMapping`. This is **inversion of control**: the framework decides when your code runs, not the other way round.

```
package com.ankit.bigtable_api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController // "this class holds web routes — scan me"
public class HelloController {
    @GetMapping("/") // "run me for GET /"
    public String helloWorld() {
        return "Hello World";
    }
}
```

`@RestController`

Marks a whole class as "contains web endpoints." Combines the job of FastAPI's `app = FastAPI()` + auto-JSON serialization.

`@GetMapping("/")`

Marks one method as the handler for GET requests to a path. Direct equivalent of `@app.get("/")`.

Annotation (`@...`)

A "sticky note" on a class/method that tells Spring what role it plays. Does nothing by itself — Spring reads it at startup.

`package`

A namespace that must match the file's real folder path exactly. Java enforces this; Python treats it as convention only.

static vs. new — when do you need an object?

The question that unlocked why `SpringApplication.run()` is called directly on the class name.

⚡ static — the light switch

Belongs to the **class itself**. No object needed — call it directly: `ClassName.method()`. Use when the action doesn't need to remember anything specific to one instance. `main()` is static because when the JVM starts, nothing exists yet to call it "on."

🔧 instance — the assembled lamp

Needs an object built first: `new ClassName()`, then call methods on *that* object. Use when each object must hold its own distinct data — e.g. two `Bigtable` rows with different data need two separate objects.

Python → Java translation table

Same concepts, stricter syntax. Refer back here whenever new Java code feels unfamiliar.

CONCEPT	PYTHON / FASTAPI	JAVA / SPRING BOOT
Install dependencies	<code>pip install fastapi</code>	<code>pom.xml</code> + Maven
App entry point	<code>main.py</code>	<code>XApplication.java</code> + <code>main()</code>
Run the server	<code>uvicorn main:app</code>	<code>mvnw spring-boot:run</code>
Define a route	<code>@app.get("/")</code>	<code>@GetMapping("/")</code>
Mark a class as routes	(implicit via app object)	<code>@RestController</code>
Config / secrets	<code>.env</code> file	<code>application.properties</code>
Dependency manifest	<code>requirements.txt</code>	<code>pom.xml</code>
Compiled cache (ignore in git)	<code>__pycache__</code> /	<code>target/</code>

Today's Windows gotchas — filed for next time

- ✓ Zip files can double-nest on extraction — always verify with `Test-Path` before trusting a path.
- ✓ New PATH entries need a **full VS Code restart**, not just a new terminal tab.
- ✓ `git add .` only reaches the current folder and below — run `git status` from the repo root to see everything.
- ✓ "Terminate batch job (Y/N)?" on Ctrl+C is a Windows `.cmd` quirk, not a Java/Spring behavior — just answer Y.

The daily Git ritual — non-negotiable

Every single day, in this order, no exceptions.

- 1 `git status` → confirm a clean tree before starting
- 2 `<do the day's work>` → code, run, verify it works
- 3 `git add .` → from the repo root
- 4 `git commit -m "Day X (Java): ..."` → descriptive, never "update"
- 5 `git push` → checkpoint saved, safe to stop

Day 1 — what actually got built

- ✓ JDK confirmed (Java 22) and Maven installed + added to PATH manually.
- ✓ Spring Boot project scaffolded via `start.spring.io`, living in `java-api/` inside the existing repo.
- ✓ Diagnosed and fixed a missing `spring-boot-starter-web` dependency by reading `pom.xml` directly.
- ✓ Confirmed embedded Tomcat starts on port 8080.
- ✓ Wrote and ran the first real endpoint — `GET /` → "Hello World" — via a separate Controller class.
- ✓ Traced a full request lifecycle end-to-end: browser → Tomcat → `DispatcherServlet` → method → response.

A class is a blueprint. new builds a real object from it.

Every Bigtable row, request, and response you'll write from here on is just this pattern, repeated.

```
public class Car {  
    private String color; // hidden – no outside class can touch this directly  
  
    public Car(String color) { // constructor – runs on new Car(...)  
        this.color = color; // "this object's" field = the value passed in  
    }  
  
    public String getColor() { return color; } // controlled read access – a "getter"  
}
```



The blueprint (class)

Defines *what fields and methods will exist* — holds no real data itself. `Car` alone creates nothing in memory until it's used.



The real thing (object)

`new Car("Red")` and `new Car("Blue")` each allocate a separate, independent block of memory — changing one never affects the other.

private + getter — why hide a field just to expose it again?

Not mainly about hiding fields. It's about controlling every read/write through one gate — so validation, logging, or internal changes can happen later without breaking callers.

If speed were public

`car1.speed = -500;` — anyone, anywhere, no validation. Broken state, nothing stops it.

With private + `setSpeed()`

Every write funnels through one method that can reject bad values — e.g. `if (newSpeed < 0) throw ...`

One URL, two independent parsing mechanisms

Tracing GET /hello/Ankit?greeting=Namaste end to end.

```
@GetMapping("/hello/{name}")
public String greet(
    @PathVariable String name,
    @RequestParam(required = false, defaultValue = "Hello") String greeting) {
    return greeting + ", " + name + "!";
}
```

@PathVariable — from the URL path

Matched against the registered pattern `/hello/{name}`. Spring captures whatever fills the `{name}` slot — here, `Ankit`.

@RequestParam — from the query string

Parsed independently as `key=value` pairs after `?`. Can be optional with `required = false` + a `defaultValue`.

Both are fully resolved by Spring **before** your method is ever called — by the time `greet()` runs, it just receives two clean, ready-to-use String arguments. Same inversion-of-control idea from Day 1, now feeding two values in instead of zero.

Why defaultValue must be literal text

Annotations are resolved entirely at **compile time** — before the program ever runs. Their contents must be values the compiler can read directly off the page.

Compiles: a literal

`defaultValue = "Hello"` — the compiler sees the exact text, no execution needed.

Fails: a variable

`defaultValue = myDefault` → *"element value must be a constant expression"* — its value only exists once the program runs.

CONCEPT	PYTHON / FASTAPI	JAVA / SPRING BOOT
Path parameter	<code>def read_item(item_id: int):</code>	<code>@PathVariable String name</code>
Optional query param + default	<code>q: str None = None</code>	<code>@RequestParam(required=false, defaultValue="...")</code>
Hide internal data	<code>(convention only – _underscore)</code>	<code>private + getter/setter (enforced)</code>
Create an object	<code>Item(name="x")</code>	<code>new Item("x")</code>

List and Map — with types locked in by generics

Later, one Bigtable row's columns will literally arrive as a `Map<String, String>` — this is that pattern, learned early.

```
import java.util.List;
import java.util.ArrayList;

List<String> names = new ArrayList<>();
names.add("Ankit");
names.get(0); // "Ankit" — no [] indexing in Java

import java.util.Map;
import java.util.HashMap;

Map<String, Integer> ages = new HashMap<>();
ages.put("Ankit", 34);
ages.get("Ankit"); // 34
```

Generics — List<String>

Declares upfront exactly what type a collection holds.

`names.add(42)` won't even compile — Python lets any type into a list; Java catches the mismatch before the program runs.

Interface vs. implementation

`List / Map` are contracts describing behavior;
`ArrayList / HashMap` are concrete implementations.
Convention: declare with the interface, build with `new SomeImplementation()`.

Autoboxing — why Integer, not int

Generics only accept object types, never raw primitives — `List<int>` won't compile. Java silently wraps 5 into an `Integer` object behind the scenes whenever it's added to a collection.

Checked exceptions: the compiler forces you to face failure

Same philosophy as private fields and generics — catch problems before the program ever runs, not after it fails in production.

Python's approach

Trusts the developer. If you forget to handle a risky operation, nothing stops you — it crashes the first time it actually happens at runtime, possibly in front of a real user.

Java's approach

For operations touching the outside world (files, network, Bigtable), the compiler **physically won't compile** your code unless you handle the failure or declare `throws` to pass it up the chain.

The enterprise-grade pattern: custom exception + `@ExceptionHandler`

Don't rely on Spring's hidden default error page (version-fragile). Take direct control of the response shape.

```
// 1. A custom exception — inherits from RuntimeException via "extends"
class PersonNotFoundException extends RuntimeException {
    public PersonNotFoundException(String message) { super(message); }
}

// 2. Thrown from your endpoint like any exception
throw new PersonNotFoundException("Person not found: " + name);

// 3. A dedicated handler builds the exact response you want
@ExceptionHandler(PersonNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public Map<String, String> handlePersonNotFound(PersonNotFoundException ex) {
    Map<String, String> error = new HashMap<>();
    error.put("error", ex.getMessage());
    return error;
}
```

extends

"This class is a specialized version of that one." Inherits everything `RuntimeException` already does, plus a meaningful name of your own.

@ExceptionHandler

Annotation-based routing again — but triggered by an exception type, not a URL. Same lookup-table idea from Day 1's request flow.

CONCEPT	PYTHON / FASTAPI	JAVA / SPRING BOOT
List	<code>names = []</code>	<code>List<String> names = new ArrayList<>()</code>
Dict	<code>ages = {}</code>	<code>Map<String, Integer> ages = new HashMap<>()</code>
Custom API error	<code>raise HTTPException(404, "...")</code>	<code>throw new CustomException(...) + @ExceptionHandler</code>
Handle an error type	<code>except SomeError:</code>	<code>catch (SomeException e) { ... }</code>

You never write "new BigtableDataClient()" — Spring does it once, for you

The mechanism behind "inversion of control," made concrete: an expensive network client, built once at startup, shared everywhere.

```
@Configuration // "this class produces beans – scan me"
public class BigtableConfig {

    @Bean(destroyMethod = "close") // close() called automatically on shutdown
    public BigtableDataClient bigtableDataClient() throws IOException {
        BigtableDataSettings settings = BigtableDataSettings
            .newBuilderForEmulator("localhost", 8086)
            .setProjectId("test-project").setInstanceId("test-instance").build();
        return BigtableDataClient.create(settings);
    }
}

@RestController
public class RowController {
    private final BigtableDataClient dataClient;

    // Spring sees this needs a BigtableDataClient, finds the bean, injects it
    public RowController(BigtableDataClient dataClient) {
        this.dataClient = dataClient;
    }
}
```



@Configuration + @Bean

Tells Spring *how* to build an object, once, at startup. Perfect for anything expensive or connection-holding — exactly the `BigtableDataClient`.



Constructor injection

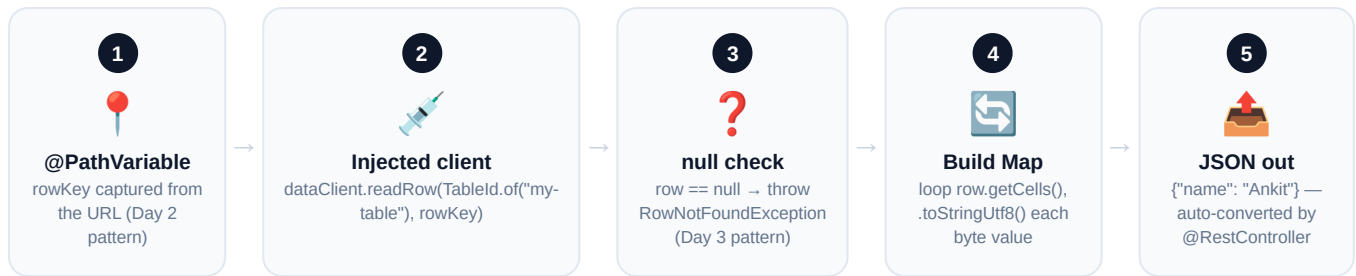
Never write `new RowController(...)` yourself. Spring sees the constructor's parameter type, finds the matching bean, and supplies it automatically.

Why not split into controller/service/repository yet?

Deliberate, not an oversight. Learning DI + beans + exception handling *and* a multi-layer refactor simultaneously is too much at once. Get it working in one file first — refactor into proper layers once the whole picture is visible.

GET /rows/{rowKey} — the full loop, finally end to end

HTTP request → Spring routing → injected Bigtable client → real emulator data → JSON response.



Cells are timestamped, confirmed live

Writing the same row/column twice did **not** overwrite — two `RowCell`s came back, newest first, different timestamps. The Bigtable theory from Python Day 4, now seen for real. No garbage-collection policy configured yet — cells accumulate unboundedly until one is set.



TableId.of("my-table") vs raw String

`readRow(String, String)` is deprecated in favor of `readRow(TableId, String)` — stronger type-safety so a table ID can never accidentally be swapped with a row key at compile time.

CONCEPT	PYTHON / FASTAPI	JAVA / SPRING BOOT
Build object once, reuse everywhere	module-level singleton / lifespan hook	@Configuration + @Bean
Supply a dependency to a class	Depends() / manual constructor arg	constructor injection (automatic)
Raw bytes → readable string	bytes.decode("utf-8")	byteString.toStringUtf8()
Cleanup on shutdown	try/finally, lifespan context	@Bean(destroyMethod = "close")

NEXT UP — DAYS 5–6

Refactor into proper controller/service/repository layers, then round out the Bigtable endpoints (writing rows via POST, listing/filtering) before moving into Spring Security for auth in Days 7–9.